

# IEEE AICAS 2026 Grand Challenge Technical Report

Team Name: aiha-hyu, Team ID: 1349573

## 1. Introduction

The AICAS 2026 Grand Challenge focuses on the efficient deployment of Qwen3-VL-2B-Instruct, a state-of-the-art multimodal large language model, for single-batch image-to-text inference. In the final round, our implementation targets Alibaba Cloud PAI platform equipped with Alibaba’s PPU (Parallel Processing Unit) Zhenwu 810E. This deployment setting requires optimization across the full inference pipeline, including model quantization, multimodal preprocessing, prefill execution, and autoregressive decoding.

Single-batch multimodal inference has different bottlenecks from conventional high-throughput batch serving. Because only one sample is processed at a time, the accelerator can be underutilized if the execution path contains frequent framework dispatch, small operator launches, synchronization points, or fragmented memory accesses. In addition, Qwen3-VL introduces vision-language-specific costs: input images must be converted into visual tokens, multimodal position IDs and RoPE-related offsets must be constructed, visual features must be injected into the language-model sequence, and the decoder must process a long multimodal context before generating the first token. Therefore, optimizing only the largest matrix multiplications is not sufficient; the entire path from image preprocessing to token generation must be considered.

In the final round, we placed particular emphasis on quantization while also improving the execution path of the model. We applied activation-aware weight scaling and GPTQ-style quantization to reduce the effective weight precision of major projection layers while preserving output quality. The resulting quantized weights are connected to optimized low-bit execution paths, including weight-only GEMV/GEMM kernels for the batch-1 decode regime. This reduces memory bandwidth pressure and improves the efficiency of linear projections, which dominate both prefill and decode computation.

Beyond quantization, we further reduce computation through structural and input-side optimizations. Layer skipping is applied to selected prefill and decode paths, including POP-based prefill skipping and early-exit-style tiny graph decoding, to reduce decoder-layer computation. Decoder MLP channel pruning reduces the intermediate dimension of the SwiGLU MLP while keeping the remaining layers dense and compatible with fused execution. The visual input pixel budget is limited to 300K pixels to control the number of visual tokens and reduce vision encoder and prefill cost. These methods lower the amount of computation that must be performed, rather than only accelerating the same computation.

We also redesign the execution path to reduce software and memory overhead. Lean vision, prefill, and decode paths bypass unnecessary PyTorch and HuggingFace module dispatch by pre-extracting weights and directly executing the required tensor operations. Torch-level fusion combines projections such as QKV and gate-up into larger dense operations, while custom fused kernels optimize repeated memory-bound operations such as normalization, RoPE, gated activation, deepstack feature injection, and decode attention. Graph-based decode replay with static buffers further reduces per-token scheduling overhead during autoregressive generation.

Finally, we introduce cache-based reuse for multimodal preprocessing. Vision feature caching avoids redundant vision encoder execution for repeated processed image tensors, while RoPE and position-ID caching reduce repeated multimodal position computation and synchronization overhead. Together, these components form an end-to-end optimized inference system that combines quantization, structural pruning, input-budget control, lean execution, fused kernels, graph replay, and cache reuse. The goal is to improve first-token latency and generation throughput on the Alibaba Cloud PAI PPU platform while maintaining compatibility with the original Qwen3-VL interface and targeting high output quality under the applied optimizations.

## **2. Lean Architecture**

To establish a high-performance inference engine, we first analyzed the systemic bottlenecks in the abstraction layers of PyTorch and HuggingFace Transformers. In single-batch scenarios, standard execution through `nn.Module.__call__` introduces significant software overhead such as repetitive hook checks, gradient mode management, and Python-level dispatching. Our profiling revealed that this management code accounts for a substantial portion of total latency in deep models like Qwen3-VL-2B-Instruct. To eliminate these redundancies, we implemented a Lean Architecture that bypasses the standard dispatcher by pre-extracting model weights into lists during initialization. This transformation allows the core execution loops to run as direct sequences of `F.linear` and custom kernel calls, significantly streamlining the code and increasing CPU scheduling efficiency.

### **2.1 Vision Encoder Phase**

To optimize Vision Encoder Phase, we refactored the 24 transformer layers into a streamlined sequence of direct `F.linear` calls and custom fused kernels. The core of this optimization lies in the aggressive caching of spatial-temporal priors; by storing RoPE based on `grid_thw`, we transformed redundant interpolations into nearly zero-overheaded lookups. Integrating `flash_attn_varlen_func` within this flattened loop further ensures highly efficient processing of variable length visual sequences. This approach maximizes the device utilization by minimizing CPU scheduling latencies.

### **2.2 Prefill Phase**

The prefill phase processes the full multimodal prompt before autoregressive decoding begins. In Qwen3-VL, this prompt contains both text tokens and visual tokens produced by the vision encoder, so the prefill path directly affects time-to-first-token (TTFT). Unlike decode, which processes one generated token at a time, prefill must process the entire multimodal sequence and prepare the KV cache for subsequent generation.

Following the Lean Architecture described above, our prefill path avoids repeatedly entering the original HuggingFace module hierarchy. During initialization, the decoder weights and configuration values required for prefill are extracted into a lightweight execution state. This includes normalization weights, fused QKV projection weights, Q/K normalization weights, output projection weights, fused gate-up projection weights, down projection weights, and final normalization weights. The prefill forward pass then executes each decoder layer as a direct sequence of tensor operations.

In the attention block, the lean path applies RMSNorm, computes Q, K, and V through a fused QKV projection, applies Q/K normalization and RoPE, updates the static KV cache, and computes prefill

attention through a FlashAttention-based path. FlashInfer kernels are used for repeated normalization-related operations such as RMSNorm and fused residual-add RMSNorm. This keeps the attention path efficient while avoiding unnecessary framework-level overhead.

In the MLP block, the lean path replaces separate gate and up projection calls with a fused gate-up projection, followed by the gated activation and down projection. Visual deepstack features are also passed through the prefill path and injected at the required decoder layers. The low-level fused kernels used for gated activation and deepstack processing are described separately in the kernel-fusion section.

### 2.3 Decode Phase

The Decode Phase extends the same Lean Architecture principles used in the Vision Encoder and Prefill Phase to the autoregressive generation process. By utilizing the pre-extracted weights and static scratch buffers in LeanDecodeState, we maintain a streamlined execution path that remains CUDA Graphs safe and free from Python-level dispatching overhead. The core performance of this phase is driven by our custom CUDA kernels detailed further in Section 5.1, which replace standard abstraction layers to handle the sequential nature of decoding. (For code review, we recommend focusing on our custom kernel modules rather than the ‘flashinfer’ paths; although the ‘flashinfer’ hook remains the codebase, we don’t use it.)

## 3. Bucketized CUDA Graphs

In a single-batch image-to-text inference environment, the standard PyTorch eager execution mode faces severe CPU-bound bottleneck. During the autoregressive decode phase, the Qwen3-VL-2B-Instruct model must launch thousands of small CUDA kernels for every single token generated. The overhead associated with scheduling and pushing these kernels to the PPU command queue often exceeds the actual PPU compute time. This disparity creates bubble periods where the PPU sits unproductive while waiting for the next CPU instruction. On the PPU, these bubbles lead to significant under-utilization of the available compute resources, directly hindering Throughput. To fundamentally address this, we implemented a Bucketized CUDA Graph system that streamlines the entire execution pipeline.

To satisfy the strict requirements of memory address invariance for CUDA Graphs, we bypassed PyTorch’s dynamic memory allocator. We pre-allocated static memory buffers for all input and output tensors and integrated a StaticCache mechanism to manage KV cache at fixed memory offsets. This architecture ensures that Accuracy is strictly preserved, as the computation remains bit-wise identical to the original model, only changing the execution mechanism. Furthermore, to avoid the memory explosion of capturing graphs for every possible sequence length, we adopted a bucket strategy. By analyzing the sequence length statistics of the dataset and pre-capturing execution paths for optimized intervals (e.g., 512, 640, 768, ...), we provide a robust balance between memory efficiency and hardware acceleration.

At runtime, the system dynamically selects the optimal bucket and triggers a single Graph replay command, replacing thousands of individual kernel launches. This shift effectively eliminates the Python interpreter’s overhead and framework-level scheduling latency, virtually removing the execution bubbles between kernels. By transitioning the performance bottleneck from CPU scheduling

to actual PPU computation, we achieved nearly theoretical maximum utilization on the PPU. This directly results in a substantial reduction in TTFT by optimizing the prefill-to-decode transition and a significant increase in Throughput, fulfilling the core performance requirements of the challenge.

## 4. Model Compression

### 4.1 Quantization

Under the single batch, the decode phase generates one token at a time, so each linear layer reduces to a matrix-vector (GEMV) operation between the weight matrix and a single activation vector. This operation is memory-bandwidth bound: the dominant cost is loading the FP16 weights from memory, not the multiply-accumulate work itself. Reducing weight precision to INT4 therefore directly attacks this bottleneck by cutting the memory traffic for these GEMV operations to roughly a quarter translating into lower per-token latency and higher decoding throughput. This is why quantization is applied to the decoder and LM head weights used in the LLM decode phase GEMV path, rather than to the prefill GEMM path, where the compute-to-memory ratio is much higher and weight-loading cost is amortized across many tokens, so 4-bit weight quantization yields comparatively little benefit there.

To realize this, the decoder and LM head weights are post-training quantized to 4-bit symmetric, groupwise weights using AWQ (Lin et al., 2023) and GPTQ (Frantar et al., 2022). The choice to combine AWQ and GPTQ follows directly from the AWQ paper, which notes that AWQ is orthogonal to GPTQ for 2-bit quantization thus we tried to apply both methods for 4-bit quantization. The two can be stacked for compounding gains rather than treated as mutually exclusive alternatives. Both methods' outputs are packed into the same format and consumed by a custom fused W4 GEMV kernel that we implement to perform the W4A16 matrix-vector multiplication directly, without unpacking weights back to FP16 and the design and implementation of this kernel are discussed in Section 5.1. Because the kernel only depends on this shared output format, it requires no modification regardless of which quantization methods are enabled.

To briefly outline the AWQ and GPTQ methods, AWQ (Lin et al., 2023) computes a per-channel scale  $s = \bar{x}^\alpha$  from the (mean-normalized) per-channel activation magnitude  $\bar{x}$ , and reparametrizes  $\bar{x} = (W \bullet \text{diag}(s)) \bullet (\text{diag}(s)^{-1} \bullet x)$ , absorbing  $\text{diag}(s)^{-1}$  into the preceding RMSNorm so the rescaling is invisible to the model.  $\alpha$  is swept over a 20-point grid  $\{1/20, \dots, 1\}$ , and the actual deployed INT4 quantizer  $Q(\cdot)$  (group=128, symmetric) is run on  $W \bullet \text{diag}(s)$  to pick

$$\alpha^* = \underset{\alpha}{\operatorname{argmin}} \left\| Q(W \bullet \text{diag}(s)) \bullet (\text{diag}(s)^{-1} \bullet x) - WX \right\|$$

,the  $\alpha$  minimizing the real round-trip quantization error. AWQ is applied to qkv\_proj and gate\_up\_proj.

GPTQ (Frantar et al., 2022) then refines the AWQ-scaled weight  $W'$  using the activation Hessian  $H = X^T \bullet X$  (dead channels neutralized, diagonal damped for stability).  $H^{-1}$  is overwritten with its Cholesky factor,  $\text{Cholesky}(H^{-1})^T$  (upper-triangular), and columns are quantized in blocks of 128: for column  $j$  with diagonal  $d_j = [H^{-1}]_{jj}$ , the rounding error  $e_j = (w_j - Q(w_j))/d_j$  is propagated to later columns via  $w_{j+1} \leftarrow e_j \bullet [H^{-1}]_{j,j+1}$ : with block-level errors similarly propagated

to subsequent blocks via the corresponding row-slices of  $H^{-1}$ . If the Cholesky factorization fails, it falls back to plain RTN. GPTQ is applied to the same projections as AWQ.

Both AWQ’s activation statistics and GPTQ’s Hessian are collected from the same calibration setup: forward hooks are registered on each decoder layer’s `input_layernorm` and `post_attention_layernorm`, and 256 calibration samples, which are image and text pairs drawn from the `lmms-lab/textvqa` dataset, are passed through the model. Beyond combining AWQ and GPTQ, which is itself a novel attempt at applying their reported orthogonality at 4-bit precision, we also varied the number of calibration samples and observed a clear accuracy recovery trend as the calibration set size increased, from 64 to 256, indicating that the calibration statistics, rather than only the choice of method, play a meaningful role in how much accuracy is preserved under 4-bit quantization.

The application order matters: AWQ first reparametrizes the weights to be activation-aware, and GPTQ then compensates the remaining rounding error of that reparametrized matrix. The two methods compose rather than compete, and both produce FP16 weights packed into the final 4-bit symmetric and groupwise format consumed by the fused GEMV kernel.

## 4.2 Layer Skipping and Early Exit

Across both prefill and decode, every decoder layer normally performs a full attention and MLP computation for every token. Layer skipping and Early exit reduces this by running only a subset of layers for a given pass. During prefill this method directly cuts the FLOPs spent on the compute-bound attention and MLP blocks across the whole prompt, and during decode it reduces the number of layer’s weights that must be streamed through the memory-bound GEMV path per token. The same underlying idea, skip layer’s attention and MLP work, is applied in two places: statically during prefill with a novel method called POP (He et al.,2026), and per-step during decode with tiny graph.

POP is based on the observation that, for a multimodal prompt, certain decoder layers contribute little to the prefill hidden states and can have their attention and MLP computation skipped. The key constraint is that decoding later reads every layer’s KV cache from the full model, so a skipped layer must still leave behind a correctly populated cache. POP satisfies this by partially executing skipped layers: it still computes the QKV projection, Q/K RMSNorm, rotary embedding, and KV cache write, everything required to populate the cache correctly, but omits the attention score computation, output projection, and the entire MLP block. Since none of these omitted computations are added back to the residual stream, the hidden state passes through the skipped layer essentially unchanged (aside from deepstack visual embedding injection where applicable, which are 0, 1, 2 layers), while its KV cache is fully valid for decode to consume. The skip set is configurable and 24, 25, and 26 layers are skipped.

The same idea is applied on the decode side. Because the lean decode path, which is described in Section 2.3, uses only static shapes and performs no allocations, it can be captured into a CUDA graph and replayed. Each captured decode graph is paired with a tiny graph: the same lean decode step but executed with only a prefix of the decoder layers before exiting early to the final norm and LM head.

## 4.3 LM Head Pruning

Separately from layer skipping, the LM head itself, the largest single weight matrix in the model due to the vocabulary dimension, is also pruned. A pruned vocabulary variant of the LM head

| Keep size | Pruned Channels (%) | Internal accuracy (%) | Throughput (tokens/s) |
|-----------|---------------------|-----------------------|-----------------------|
| 6144      | 0.00                | 86.36                 | 288.26                |
| 5888      | 4.17                | 85.23                 | 292.19                |
| 5760      | 6.25                | 84.09                 | 296.04                |
| 5632      | 8.33                | 87.50                 | 298.43                |
| 5504      | 10.42               | 84.09                 | 302.26                |
| 5376      | 12.50               | 84.09                 | 304.61                |
| 5248      | 14.58               | 82.95                 | 307.23                |
| 5120      | 16.67               | 80.68                 | 309.58                |
| 4864      | 20.83               | 75.00                 | 310.31                |
| 4608      | 25.00               | 68.18                 | 319.29                |
| 4352      | 29.17               | 46.59                 | 325.18                |

**Table 1.** Internal accuracy and throughput sweep of decoder MLP channel pruning on the kernel-optimized baseline (measured in NVIDIA A100 40GB)

is construed by keeping only a calibrated subset of output channels most likely to be needed by referencing the content in the json file, and this smaller head is used in place of the full LM head during the tiny graph decode steps described in Section 4.2, further reducing the GEMV size on the decode path’s most expensive projection.

#### 4.4 Decoder MLP Channel Pruning

The decoder MLP accounts for a substantial portion of the computation in Qwen3-VL-2B-Instruct. Each decoder layer contains a gated SwiGLU MLP, where the hidden state is projected into two intermediate branches: the gate branch and the up branch. The gated activation is then projected back to the hidden dimension through the down projection. Since this block is repeatedly executed across all prompt tokens during prefill and once per generated token during decode, reducing the MLP intermediate dimension can lower both prefill and decode cost.

In the original model, the decoder MLP intermediate size is 6144. We apply structural channel pruning to reduce this dimension to 5632, corresponding to an 8.33% reduction in intermediate channels. This keep size was selected as the final operating point based on an internal pruning-ratio sweep. The 5632 setting improves throughput over the unpruned baseline and achieves the highest measured internal accuracy in the sweep. More aggressive pruning ratios provide higher throughput, but they remove substantially more intermediate channels and show a clear accuracy drop beyond moderate pruning levels.

We conducted the sweep on top of the kernel-optimized baseline while changing only the MLP keep size. Accuracy was computed using our internal normalized answer-matching script. The result shows that 5632 provides the best balance between accuracy and throughput for the final submission.

The values in Table 1 were measured for pruning-ratio selection on the kernel-optimized baseline and should not be interpreted as the final end-to-end submission result. Based on this sweep, we selected 5632 as the final keep size because it improves throughput from 288.26 to 298.43 tokens/s while maintaining the highest measured internal accuracy.

The keep indices are selected offline using 64 TextVQA calibration samples. During final evaluation,

the submission does not perform calibration, retraining, or iterative pruning. Instead, it loads the precomputed keep-index file and applies deterministic structural slicing to the decoder MLP weights.

Our pruning score is a Wanda-style criterion adapted to the SwiGLU MLP structure. For each decoder layer, we run calibration forward passes and compute the SwiGLU contribution of each intermediate channel:

$$contrib_j = SiLU(gate_j) \bullet up_j$$

For mean-absolute aggregation, the activation score of channel  $j$  is computed over all calibration tokens:

$$a_j = mean(|contrib_j|)$$

To reflect how strongly each intermediate channel affects the final MLP output, we multiply this activation score by the L2 norm of the corresponding down-projection column:

$$s_j = a_j \bullet \|W_d[:, j]\|_2$$

Here, the term  $\|W_d[:, j]\|_2$  denotes the L2 norm of the  $j$ -th column of the down projection. Channels with large scores are both active on the calibration inputs and strongly connected to the output hidden dimension. For each decoder layer, we keep the top 5632 channels according to this score.

This pruning removes the same intermediate channels consistently from the gate, up, and down projections. The gate and up projections remove output rows, while the down projection removes the corresponding input columns. As a result, the MLP remains a dense computation with a smaller intermediate width, instead of becoming a sparse or runtime mask-based operation.

This property is important for inference efficiency. Because the pruned MLP still consists of regular dense linear layers, it remains compatible with fused gate-up projection, fused gated activation, quantized linear kernels, and the lean prefill/decode execution paths. Thus, the pruning reduces the amount of computation while preserving the optimized execution structure of the final inference pipeline.

## 5. Kernel Fusion and Torch-level Operator Fusion

### 5.1 Kernel Fusion

**fused\_layernorm.** LayerNorm is invoked repeatedly in the vision encoder and becomes a non-negligible latency source despite its relatively simple arithmetic. The baseline implementation typically involves multiple internal steps for statistics computation and affine transformation, with substantial global memory traffic.

To address this, we implemented a custom fused LayerNorm kernel optimized for FP16 execution. The kernel uses vectorized float4 loads, corresponding to 8 FP16 values per transaction, FP32

---

**Algorithm 1** Decoder MLP Channel Pruning

---

```
1: Input:
2:   Decoder MLP weights:  $W_g, W_u, W_d$ 
3:   Calibration inputs:  $\mathcal{D}$ 
4:   Target keep size:  $k = 5632$ 
5: for each decoder layer do
6:   Run calibration forward passes on  $\mathcal{D}$ .
7:   for each calibration token hidden state  $\mathbf{x}$  do
8:      $gate = W_g \mathbf{x}$ 
9:      $up = W_u \mathbf{x}$ 
10:     $contrib = \text{SiLU}(gate) \cdot up$ 
11:   end for
12:   Accumulate channel-wise mean-absolute activation:
13:    $a_j = \text{mean}(|contrib_j|)$ 
14:   Compute down-projection column norm:
15:    $d_j = \|W_d[:, j]\|_2$ 
16:   Compute final importance score:
17:    $s_j = a_j \cdot d_j$ 
18:   Select keep indices:
19:    $\mathcal{K} = \text{top-}k \text{ indices of } \mathbf{s}, \text{ where } k = 5632$ 
20:   Apply structural slicing:
21:   gate_proj: keep rows  $\mathcal{K}$ 
22:   up_proj: keep rows  $\mathcal{K}$ 
23:   down_proj: keep columns  $\mathcal{K}$ 
24:   Save  $\mathcal{K}$  to the keep-index file.
25: end for
26: Output:
27:   Structurally pruned dense decoder MLP
```

---

accumulation for numerical stability, warp-level reduction using shuffle primitives, block-level reduction through shared memory and in-kernel affine transform using weight and bias.

Each block processes one row. The kernel first computes the row-wise mean and variance, then directly applies normalization and affine transformation without materializing unnecessary intermediate tensors.

The implementation is tuned for the vision hidden size used in the model. For hidden dimension 1024, a block size of 128 threads provides a favorable balance between parallelism, occupancy, and shared memory usage.

**fused\_rmsnorm.** RMSNorm appears in two major forms in the model: decoder hidden-state normalization and smaller per-head normalization paths such as q/k normalization. These paths have very different hidden dimensions, so a single block configuration is not optimal for both.

Our fused RMSNorm kernel uses vectorized float4 loads, FP32 accumulation of sum-of-squares, warp and block reduction, in-kernel scaling by learned weight.

Unlike a generic implementation, our kernel dispatch is shape-aware. Smaller hidden sizes, such as 128, use a smaller block size to maximize occupancy and reduce idle threads. Larger hidden sizes, such as 2048, use a larger block size to distribute work efficiently across more threads.

This design is important because normalization is highly memory-bound. Overprovisioning threads for small vectors wastes scheduling capacity, while underprovisioning threads for large vectors increases reduction cost. By matching block size to hidden dimension, we improve the execution efficiency of both normalization regimes.

**fused\_rope\_ver6.** Rotary Position Embedding (RoPE) is applied very frequently in the attention path and is often implemented through a combination of tensor reshaping, broadcasting, slicing, and elementwise arithmetic. Although each individual operation is simple, the full sequence introduces extra memory traffic and kernel-launch overhead.

We therefore implemented specialized fused RoPE kernels. The design supports both text RoPE, where tensors follow the [batch, heads, seq, dim] layout, and vision RoPE, where tensors follow the [seq, heads, dim] layout.

The text kernel directly reads the low and high halves of the head dimension, applies the rotary transform, and writes results back in vectorized half2 form. This reduces or avoids repeated cosine/sine expansion across heads by broadcasting within the kernel.

The vision kernel uses a different row-mapping strategy tailored to the visual tensor layout. Work is partitioned so that each block processes multiple (sequence, head) rows, and each lane handles a pair of channels. The layout-aware specialization reduces unnecessary index computation and improves data locality.

The key design choice here is that we do not treat all RoPE applications as one generic tensor program. Instead, we exploit the actual tensor shapes in the text and visual branches and implement separate execution paths accordingly.

**fused\_gelu.** The vision MLP repeatedly applies GELU activation, which is an elementwise operation but is invoked many times across visual transformer blocks. On the PPU, such activations are usually memory-bound, so the main opportunity lies in reducing overhead rather than changing arithmetic complexity.

Our fused GELU kernel uses the tanh-based GELU approximation already adopted by the model and applies it in a vectorized half2 manner. Each thread processes two FP16 elements at a time, converts them to FP32 for computation, applies the GELU formula, and converts the result back to FP16.

Although GELU alone is not the largest latency source, it is a recurring component in the visual encoder. Since GELU appears repeatedly across vision blocks, reducing its per-call overhead contributes to cumulative encoder-side speedup.

**fused\_silu\_mul\_ver2.** The decoder MLP uses a gated activation structure in which the gate branch is activated with SiLU and then multiplied elementwise by the up branch. In a straightforward implementation, this requires at least two separate steps:

1. compute SiLU on the gate tensor,
2. multiply the result with the up tensor.

This creates additional intermediate tensors and introduces extra memory traffic. Since the operation is elementwise and heavily memory-bound, eliminating redundant reads and writes is highly beneficial.

Our fused kernel assumes that the gate and up activations are stored contiguously in a single concatenated tensor, then directly reads the gate part and the up part, applies SiLU to the gate values, multiplies them with the corresponding up values, and writes the fused output.

The kernel uses half2 vectorization so that each thread processes two FP16 values at a time. This improves effective bandwidth utilization and reduces per-element overhead.

**fused\_deepstack\_process.** Our profiling revealed that the typical approach, involving Boolean masking and indexing, triggers multiple redundant CUDA kernels, such as ‘DeviceSelectSweepKernel’ and frequent Device-to-Host memory copies for size calculations, which introduce significant synchronization idle time. We optimized this by exploiting the deterministic structure of the input sequence (‘system prompt + vision tokens + text tokens’), allowing us to replace dynamic masking with a fixed ‘start\_offset’ (vision tokens appear after fixed size of system prompt in every sample) and direct in-place memory access. The resulting CUDA kernel utilizes half2 vectorized operations to perform elementwise addition of visual embeddings directly into the pre-allocated hidden states buffer. This bypasses all intermediate indexing overhead and memory allocations.

**fused\_attn\_decode\_v3.** The proposed implementation optimized the single-token inference path by restructuring the standard decoder execution flow into a two-stage fused architecture. By consolidating multiple operations that are traditionally memory-bound, we significantly reduce the kernel launch overhead and global memory traffic, which are the primary bottlenecks.

For NRC fusion, we implemented a unified kernel that fuses RMSNorm, RoPE, and KV cache management. The key idea lies in a register-level data path where feature normalization and positional encoding are computed in-place before the results are committed to the cache. This approach minimizes intermediate DRAM R/W cycles and leverages intra-warp communication for reductions, ensuring the compute units remain saturated.

For Tiled GQA (Grouped Query Attention), the attention mechanism is redesigned using an adaptive tiling strategy that dynamically adjusts its parallelization granularity based on the KV cache length. To overcome the synchronization bottlenecks of traditional multi-pass attention, we utilized an atomic cross-tile reduction pattern, allowing a single kernel to handle both local tile computation and global result merging. Furthermore, the implementation incorporates hardware-native math intrinsics and vectorized memory access to maximize the effective bandwidth of the PPU’s memory subsystem, achieving near-theoretical peak performance for Grouped Query Attention (GQA).

**fused\_w4\_gemv.** To translate the 4-bit weight format into an actual speedup, we implement custom CUDA kernels that compute the decode-time matrix-vector product directly on packed int4 weights, dequantizing on the fly in registers rather than materializing a dense FP16 weight matrix in memory. Since the batch=1 decode regime is strongly memory-bandwidth bound, storing weights at 4 bits and avoiding any intermediate FP16 expansion translates almost directly into a proportional reduction in weight-loading cost.

The kernels share a common structure: each output row is computed by a group of threads that cooperatively stream through the packed weights, decode them, accumulate the dot product in higher precision, and combine partial results via a standard reduction. On top of this base structure,

we implement several variants that differ in their memory-access pattern and data-reuse strategy, targeting different bottlenecks (raw bandwidth, cache pollution, activation reuse across multiple output rows, and batched evaluation for speculative decoding). We also implement a fused kernel that combines the SwiGLU activation with the down-projection GEMV, avoiding extra intermediate tensors and memory roundtrips.

Because no single variant or configuration is optimal across all weight shapes, kernel selection is handled by a dispatcher that picks the best-performing variant and configuration per projection, based on offline profiling on the target device, with sensible fallbacks for shapes outside the profiled set. The kernels also provide an in-place execution mode that is compatible with CUDA graph capture and are integrated into the model via a linear-layer wrapper that transparently routes the batch=1 decode path through these custom kernels while falling back to standard dense computation otherwise.

## 5.2 Torch-level Operator Fusion

**fused\_qkv.** To further minimize computational latency, we consolidated the disparate query, key, and value projections into a single ‘FusedQKVProjection’ layer. This refactoring replaces three independent ‘nn.Linear’ calls with a single matrix multiplication, effectively doubling the computational intensity and efficient memory access patterns. Following the fused projection, the output tensor is partitioned into Q, K, and V components, where query and key states pass per-head RMSNorm before the application of RoPE. This approach not only reduces the number of kernel launches but also aligns with our Lean execution strategy by facilitating more efficient data movement during the transformer layer’s forward pass.

**fused\_gate\_up.** In the original MLP design, the gate projection and up projection are computed by two separate linear layers, even though both consume the same input hidden state. This causes redundant memory reads of the input and incurs two independent projection launches.

To remove this redundancy, we concatenate the two projection weights into a single larger weight matrix and replace the original pair of linear operators with one fused projection  $x \rightarrow [\text{gate}, \text{up}]$ .

The fused output is then passed to fused\_silu\_mul, which applies the gated activation efficiently, followed by the original down projection.

This optimization restructures the execution graph while preserving the mathematical equivalence of the original gate/up projection and gated activation computation. Instead of performing  $g = xW_g^T$ ,  $u = xW_u^T$ ,  $y = \text{SiLU}(g)u$ , we preform  $[gu] = x[W_g; W_u]^T$ ,  $y = \text{fused\_SiLU\_Mul}([gu])$ .

The main benefit is that the shared input is loaded only once for the two projections, and the number of major operator launches in the MLP front half is reduced.

## 6. Additional Methods

### 6.1 Linear Patch Embedding

The original visual patch embedding uses a Conv3d operator with kernel size equal to stride. In this configuration, the convolution does not perform overlapping sliding-window aggregation. Instead, it

effectively extracts non-overlapping spatio-temporal patches and applies a linear projection to each patch.

Based on this observation, we replaced the original Conv3d patch embedding with a standard Linear layer initialized by reshaping the original convolution weight. The original convolution kernel is reshaped from  $[C_{out}, C_{in}, T_p, P, P]$  into  $[C_{out}, C_{in} \cdot T_p \cdot P \cdot P]$ , so that each flattened patch is projected by a single matrix multiplication.

This replacement provides two benefits. First, it preserves exact mathematical equivalence because the original operation is already a non-overlapping patch projection. Second, it avoids the extra cuDNN-related setup and dispatch overhead associated with Conv3d, especially for inference-time small or moderately sized workloads.

## 6.2 Input Image Pixel Budget Adjustment

The number of visual tokens in Qwen3-VL is determined by the image preprocessing stage. Since visual tokens are later processed by both the vision encoder and the language-model prefill path, large input images can increase the cost of the entire multimodal inference pipeline.

To limit this cost, we set the maximum image pixel budget to 300K pixels in the final submission. This is applied through the processor configuration by setting the image processor’s maximum pixel value. Images above this budget are constrained by the processor before they are converted into visual tokens. The model architecture and decoder execution path remain unchanged.

This adjustment is a simple input-side optimization. It reduces the number of visual tokens generated from large images, which lowers vision encoder workload and shortens the multimodal sequence processed during prefill. Because the change is applied before model execution, it is complementary to the other optimizations such as quantization, MLP channel pruning, lean prefill, fused kernels, and graph-based decode replay.

## 6.3 Redundant Computation Elimination via Caching

Multimodal inference in Qwen3-VL includes preprocessing steps that can be repeated for identical processed image tensors or repeated grid configurations. In particular, image features must be extracted from processed pixel tensors, and multimodal position IDs with RoPE-related offsets must be constructed before the language model prefill stage. Although these operations are not the largest matrix multiplications in the model, they can still add overhead in a single-batch inference pipeline.

To reduce redundant vision computation, we introduce a vision feature cache around the image feature extraction path. The cache wraps the original `get_image_features` function and stores its output for previously seen processed image tensors. The cache key is derived from the tensor shape, dtype, and sampled tensor values, rather than from the memory pointer alone. This avoids unsafe cache hits caused by memory reuse while keeping the lookup lightweight. On a cache hit, the system reuses the previously computed vision outputs, including both the pooled image features and deepstack features.

We also cache RoPE-related multimodal position computation. For a given combination of image tensor, input IDs, attention mask, multimodal token type IDs, and image/video grid information, the system builds a content-aware cache key and stores the computed position IDs and RoPE deltas. When the same structure appears again, the cached values can be reused instead of calling the full

multimodal RoPE-index computation path again. This reduces repeated host-device synchronization and tensor construction overhead before prefill.

A smaller cache is also used inside the lean vision encoder for grid-dependent positional and rotary embeddings. Since these embeddings are determined by the image grid shape, caching them by `grid_thw` avoids recomputing the same positional and rotary tensors for repeated grid configurations.

These cache-based optimizations do not change the model architecture or the decoder computation. They only reuse deterministic intermediate results from the multimodal preprocessing path. Therefore, they are complementary to the main execution optimizations such as quantization, MLP channel pruning, lean prefill, fused kernels, and graph-based decode replay.

## 7. Result

| Metric                   | Baseline | Optimized<br>(Preliminary) | Optimized<br>(Final) | Improvement<br>(vs. Baseline) | Improvement<br>(vs. Preliminary) |
|--------------------------|----------|----------------------------|----------------------|-------------------------------|----------------------------------|
| Accuracy                 | 0.7089   | 0.6889                     | 0.7133               | +0.44pp higher                | +2.44pp higher                   |
| TTFT<br>(ms)             | 77.28    | 46.39                      | 16.44                | 4.70x lower                   | 2.82x lower                      |
| Throughput<br>(tokens/s) | 55.22    | 357.55                     | 5419.08              | 98.1x higher                  | 15.2x higher                     |

**Table 2.** End-to-End performance comparison on the Alibaba Zhenwu 810E PPU

In the preliminary round, only system-level optimizations were applicable, since algorithm-level changes such as token pruning or weight modification were not yet supported. The preliminary submission therefore combined a Lean Architecture, bucketized CUDA Graphs, CUDA kernel fusion, torch-level operator fusion, and a linear patch embedding for the vision encoder, developed on an NVIDIA A100 GPU. For the final round, the same submission was measured on the target Alibaba Zhenwu 810E PPU. With these system-level optimization alone, average TTFT was reduced from 77.28ms to 46.39ms, which is 1.67x faster, and throughput increased from 55.22 to 357.55 tokens/s, which is 6.47x higher, while accuracy decreased slightly from 0.7089 to 0.6889.

Building on this foundation, the final submission additionally applies algorithm-level optimizations: AWQ + GPTQ 4-bit weight quantization paired with a custom fused W4 GEMV kernel, layer skipping (POP during prefill and tiny graph during decode), and LM head and MLP channel pruning. With these combined optimizations, the final submission improves all three metrics over both the baseline and the preliminary submission: TTFT is reduced to 16.44ms, which is 4.70x over baseline and 2.82x over preliminary, throughput increases to 5419.08 tokens/s, which is 98.1x over baseline and 15.2x over preliminary, and accuracy improves to 0.7133, which is +0.44pp over baseline and +2.44pp over preliminary.

## 8. Conclusion

In this work, we designed and implemented a combined algorithm- and system- level optimization scheme for Qwen3-VL-2B-Instruct inference under single-batch, fixed model deployment setting, targeting the prefill and decode phases of multimodal generation. On the system side, we build a Lean Architecture that bypasses framework dispatch overhead through pre-extracted weights, fused kernels, and CUDA graph captured execution across the decode paths. On top of this, we applied algorithm-level optimizations: AWQ + GPTQ 4-bit weight quantization paired with a custom GEMV kernel, layer skipping applicable each to prefill and decode, and weight channel pruning for LM head and MLP.

Experimental results on the target device PPU show that these optimizations preserve, and slightly improve, accuracy while delivering substantial end-to-end gains: average TTFT was reduced from 77.28ms to 16.44ms (4.70x lower first-

token latency) and average throughput increases from 55.22 to 5419.08 tokens/s (98.1x higher generation throughput) relative to the unoptimized baseline, with almost unchanged accuracy.

These results show that the two phases of VLM inference call for fundamentally different optimization strategies: prefill, being compute-bound over a long token sequence, benefits most from reducing the amount of computation performed (layer skipping and weight channel pruning), while decode, being memory-bandwidth-bound, benefits most from reducing the volume of weight data moved per token (4-bit weight quantization with fused GEMV kernel) and early exit with tiny graph. Because these techniques operate on different axes over weight precision vs. layer count and different phases over prefill vs. decode, they compose without conflict and layering them on top of a lean architecture yields end-to-end gains beyond what any single technique provides in isolation.

## References

- [1] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guanxuan Xiao, Xingyu Dang, Chaung Gan, Song Han. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration, 2023.
- [2] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers, 2022.
- [3] Junhui He, Zhihui Fu, Jun Wang, Qingan Li. POP: Prefill-Only Pruning for Efficient Large Model Inference, 2026.
- [4] Mingjie Sun, Zhuang Liu, Anna Bair, J. Zico Kolter. A Simple and Effective Pruning Approach for Large Language Models, 2023.